MINISTRY OF EDUCATION AND SCIENCE OF UKRAINE
NATIONAL TECHNICAL UNIVERSITY
“KHARKIV POLYTECHNIC INSTITUTE”
DEPARTMENT OF SOFTWARE ENGINEERING AND MANAGEMENT INFORMATION TECHNOLOGIES

Report of laboratory work № 3

Discipline «Software Engineering»
Theme: Planning of System Activities

Executed by:
Student of group KN-226e
Budkov O. Yu.

Checked by:
Associate professor
Melnyk K. V.

Kharkiv
2026

Theme: planning of system activities.

Goal: to obtain basic experience and knowledge to work with a project tracking system.

Tasks:

– register on the free github.com account;

– create a new repository;

– add a new user as a collaborator to the repository;

– create milestones and issues according to the previous labs and assign tasks to the users of the team;

– work with the git repository;

– prepare the report of the work.

Progress of work

Task 3.1 Registering on github.com

GitHub is a web platform for version control and teamwork. It stores the repository of the project with the full history of changes and gives tools for planning the work: milestones, issues, labels and assignees. A free account was registered on https://github.com/signup (Figure 3.1).

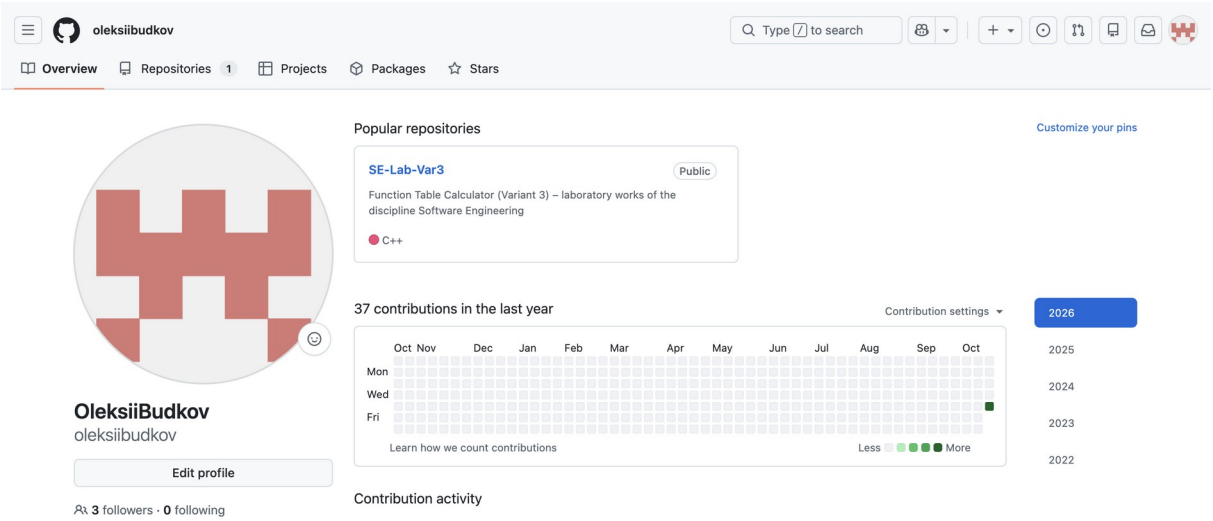


Figure 3.1 – Account on github.com

Task 3.2 Creating a New Repository

A new public repository SE-Lab-Var3 was created with the button “New” (Figure 3.2). Its description is “Function Table Calculator (Variant 3) – laboratory works of the discipline Software Engineering”. The repository stores the C++ program, the reports and the diagrams of all labs (Figure 3.3); the file README.md describes the program and the structure of the repository.

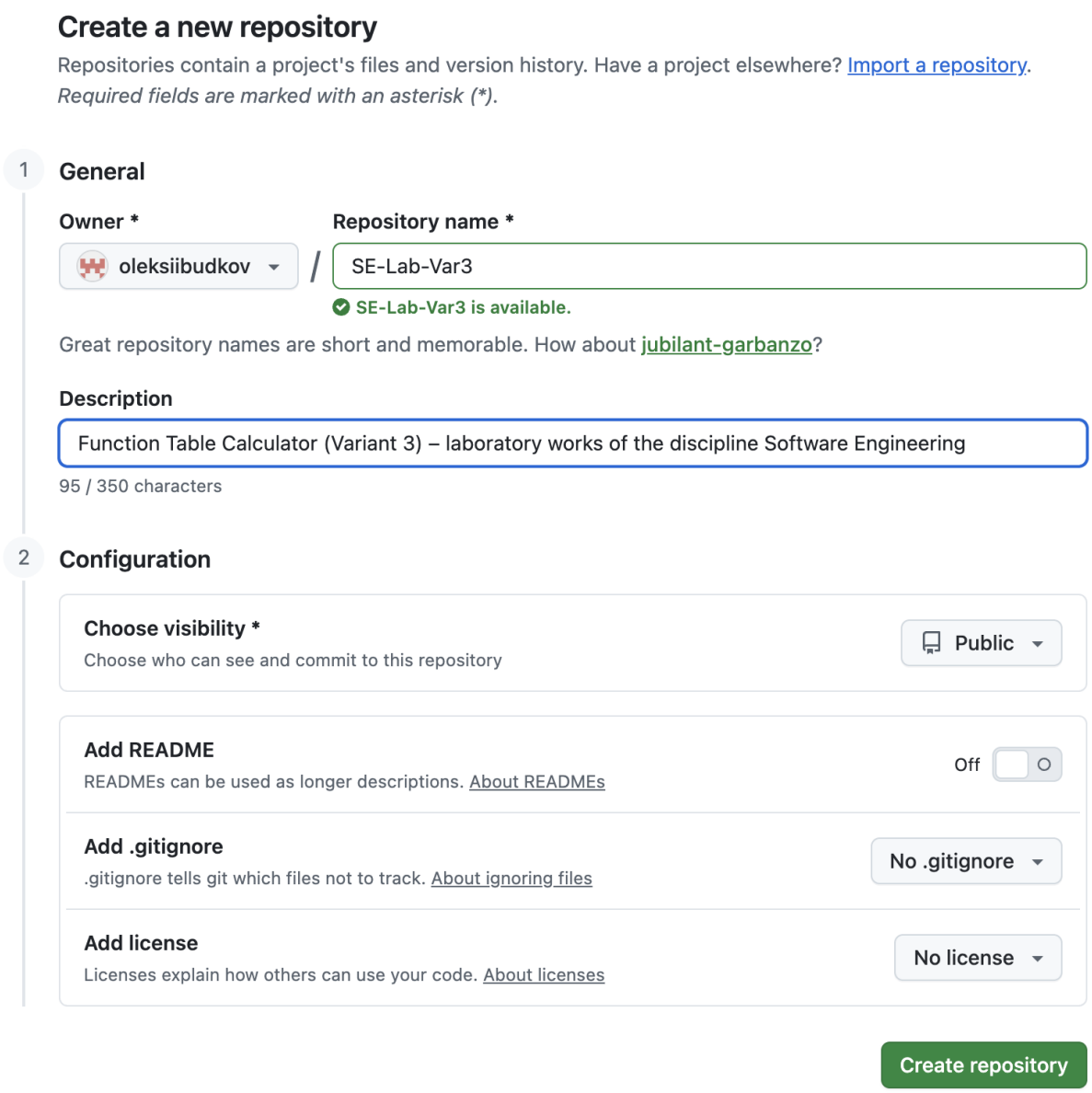


Figure 3.2 – Creating of a new repository

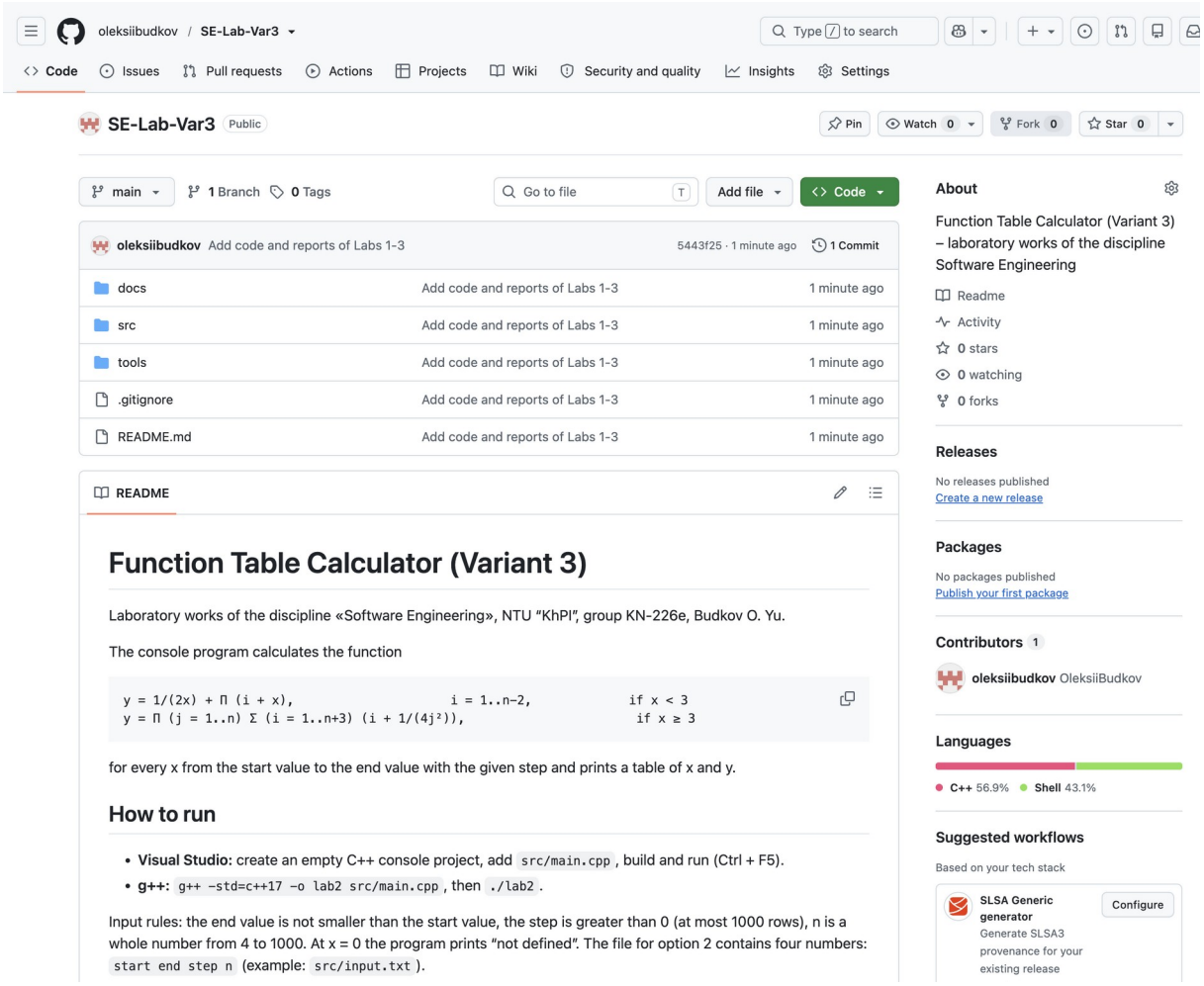


Figure 3.3 – The new repository on github.com

Task 3.3 Adding a Collaborator

The project is developed by one person, so no second member was invited. A collaborator is invited on the page Settings → Collaborators → Add people (Figure 3.4): after the invitation is accepted, the collaborator can push changes to the repository and can be assigned to issues. All issues of the project are assigned to the author of the repository.

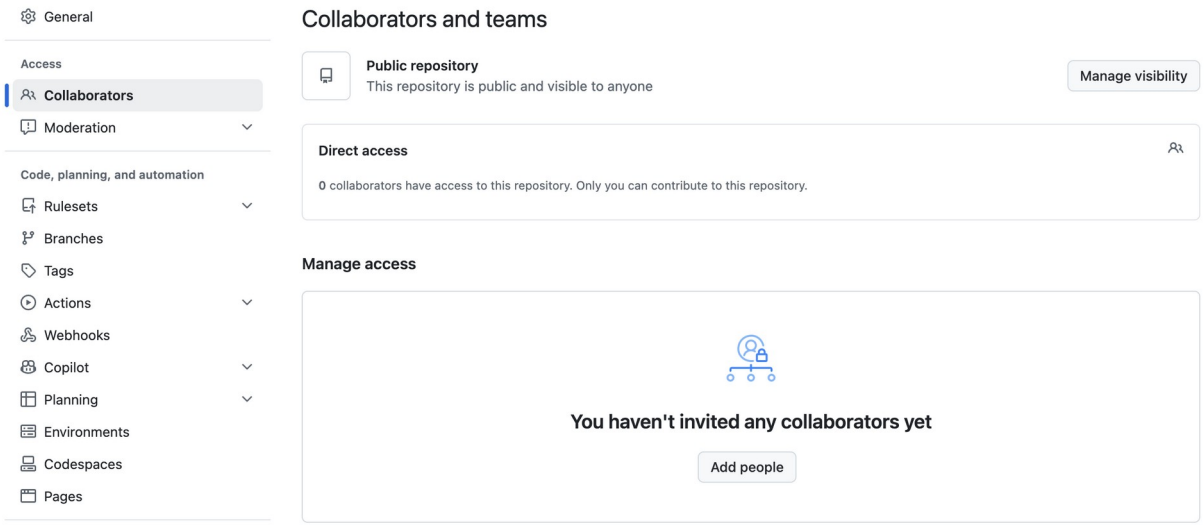


Figure 3.4 – Collaborators of the repository

Task 3.4 Milestones and Issues

For planning, an iterative life cycle model was chosen: every laboratory work is one iteration that adds a new part of the system and its documentation, and the next iteration uses the results of the previous one. So every lab is one milestone with a due date, and every milestone has one epic task and several subtasks.

Two labels were created: Epic for the big task of a milestone and SubTask for its parts (Figure 3.5). Then five milestones were created, one for every lab (Figures 3.6–3.7). For every milestone an epic task was created (Figure 3.8), and for every epic – a set of subtasks with the label SubTask, the milestone and the assignee (Figure 3.9). The epic task contains the checklist of its subtasks, so its progress is seen at once. The first milestone, the labels and the first issues were created in the web interface; the other issues were created in the same way with the GitHub CLI script tools/github_setup.sh from the repository.

All tasks are assigned to the author of the repository, who makes the analysis, the code, the diagrams, the reports and the planning. The finished tasks of Labs 1 and 2 are closed, so their milestones show 100 % progress (Figure 3.10). The subtasks of all milestones are shown in Tables 3.1–3.5.

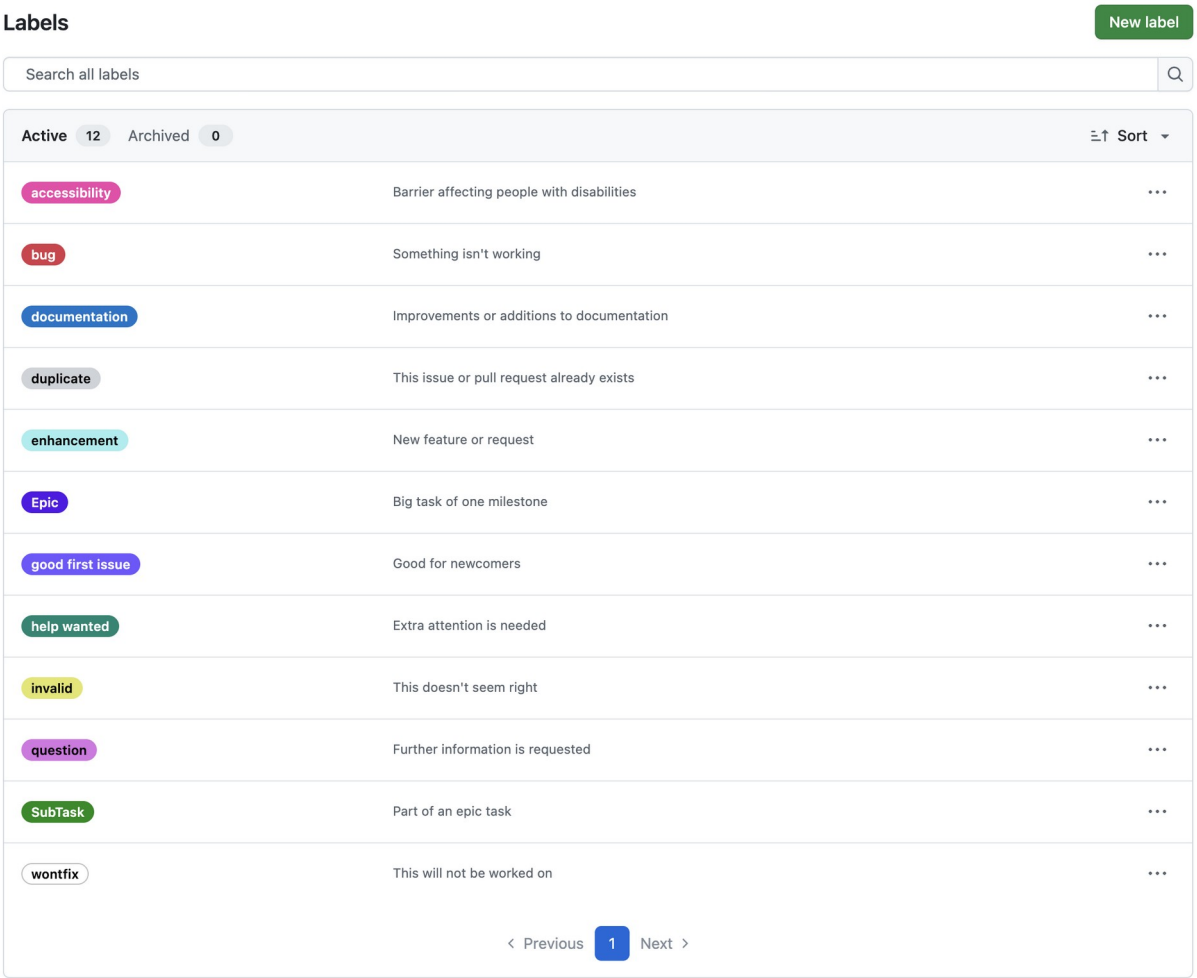


Figure 3.5 – Labels Epic and SubTask

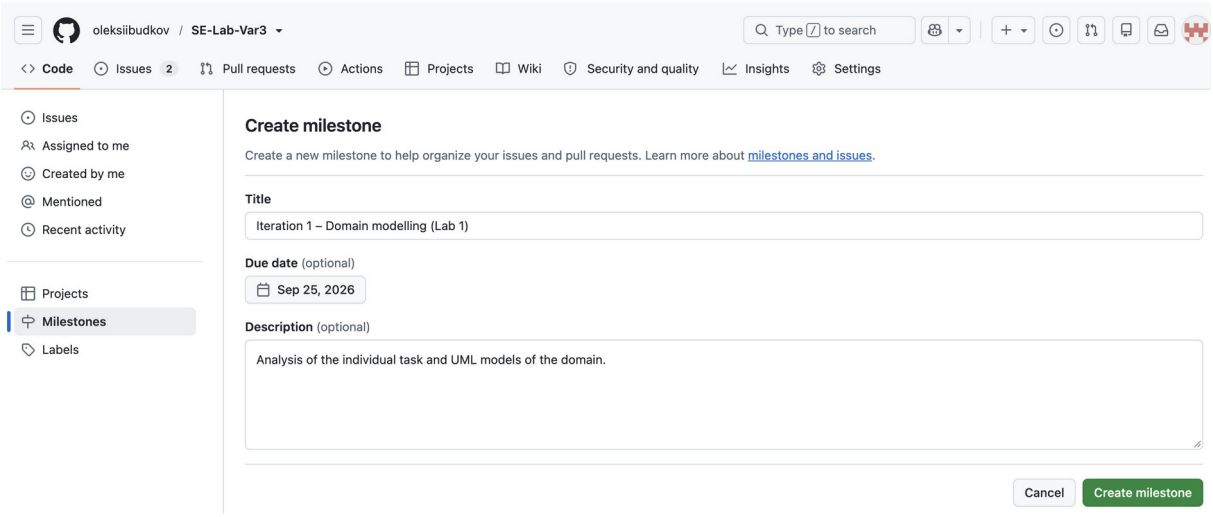


Figure 3.6 – Creating of a new milestone

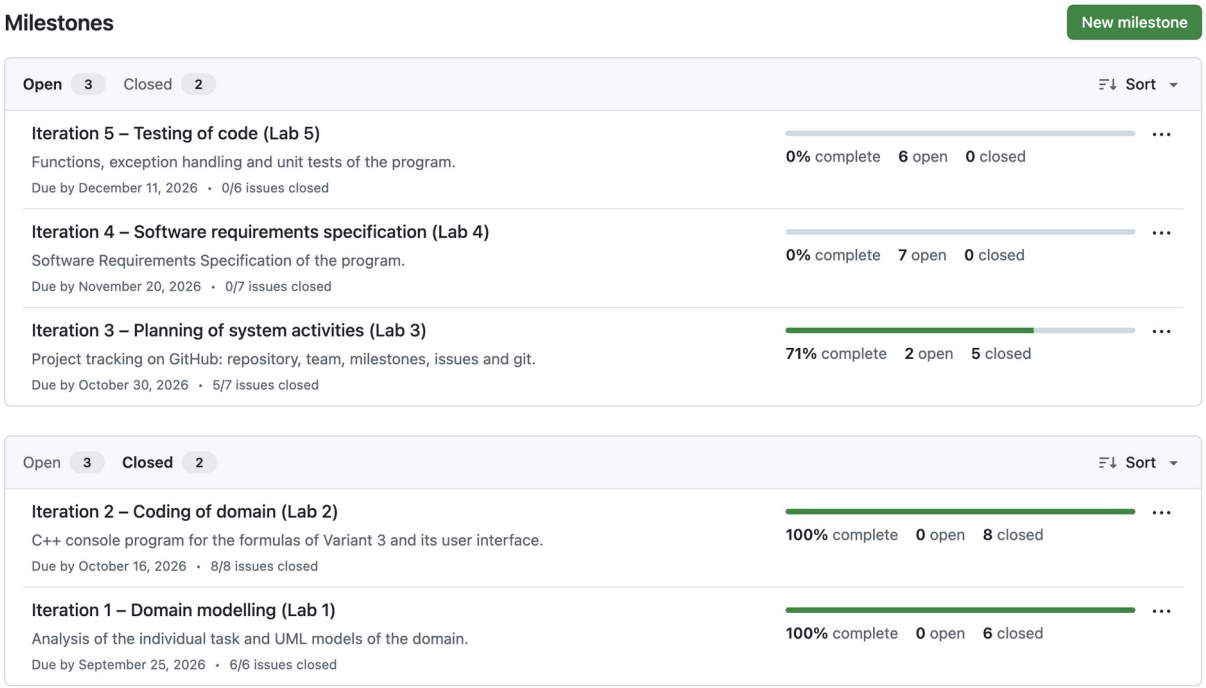


Figure 3.7 – Milestones of the project

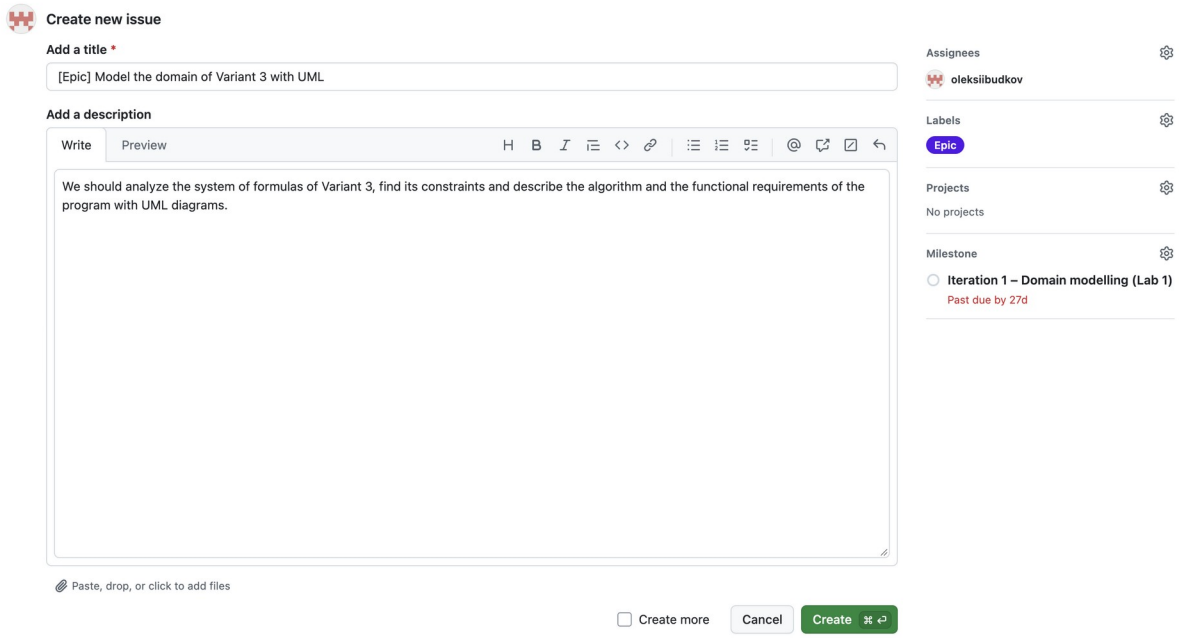


Figure 3.8 – Creating of a new epic task

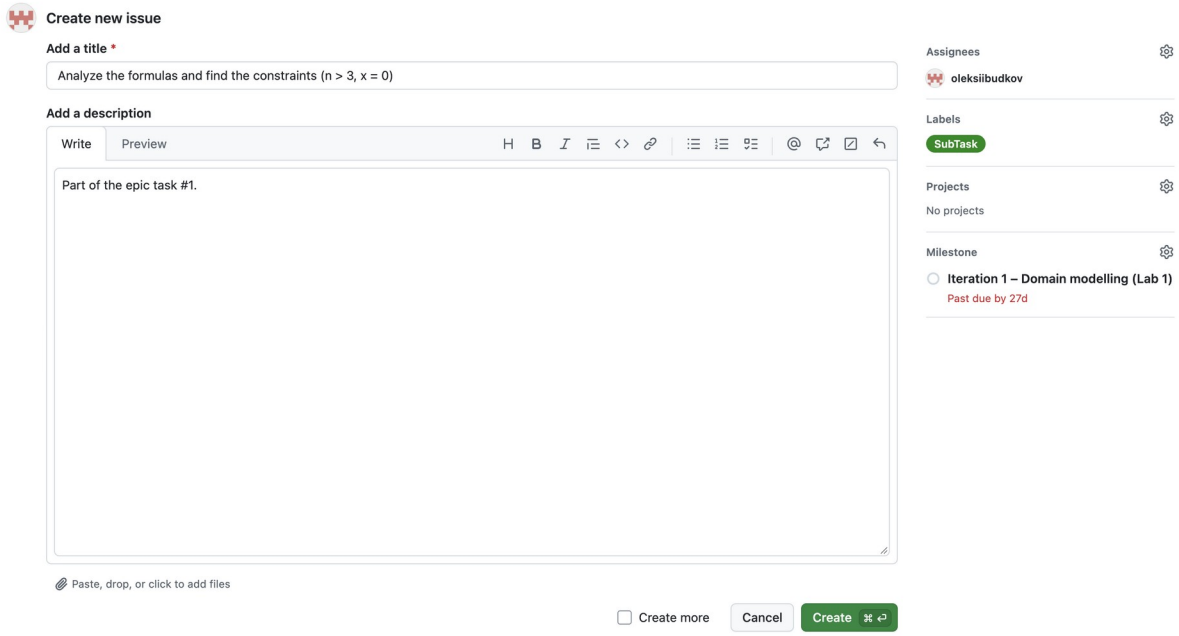


Figure 3.9 – Creating of a new subtask

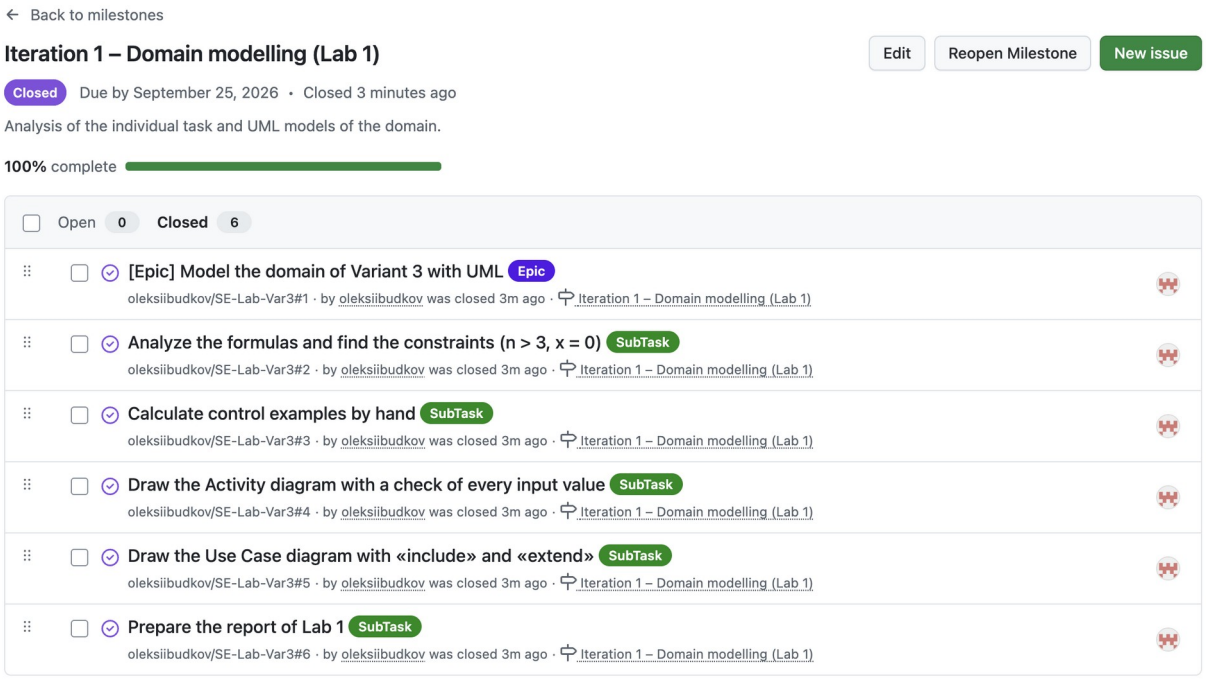


Figure 3.10 – Result table for the first milestone

Table 3.1 – Iteration 1 – Domain modelling (Lab 1), due 25.09.2026

Epic	Subtask	Assignee	Status
Model the domain of Variant 3 with UML	1. Analyze the formulas and find the constraints (n > 3, x = 0)	Budkov O.	Done
	2. Calculate control examples by hand	Budkov O.	Done
	3. Draw the Activity diagram with a check of every input value	Budkov O.	Done
	4. Draw the Use Case diagram with «include» and «extend»	Budkov O.	Done
	5. Prepare the report of Lab 1	Budkov O.	Done

Table 3.2 – Iteration 2 – Coding of domain (Lab 2), due 16.10.2026

Epic	Subtask	Assignee	Status
Implement the function table calculator in C++	1. Implement the calculation of y for both branches	Budkov O.	Done
	2. Implement keyboard input with a separate check of every value	Budkov O.	Done
	3. Implement loading of data from a file	Budkov O.	Done
	4. Implement the result table and saving to results.txt	Budkov O.	Done
	5. Add the main menu and the Help screen	Budkov O.	Done
	6. Create the wireframe and the diagram of screen transitions	Budkov O.	Done
	7. Prepare the report of Lab 2	Budkov O.	Done

Table 3.3 – Iteration 3 – Planning of system activities (Lab 3), due 30.10.2026

Epic	Subtask	Assignee	Status
Set up project tracking on GitHub	1. Create a GitHub account and a new repository	Budkov O.	Done
	2. Add a collaborator to the repository	Budkov O.	Done

Epic	Subtask	Assignee	Status
	3. Create milestones and labels Epic and SubTask	Budkov O.	Done
	4. Create epics and subtasks for all labs and assign them	Budkov O.	Done
	5. Push the source code and reports to the repository	Budkov O.	Done
	6. Prepare the report of Lab 3	Budkov O.	Open

Table 3.4 – Iteration 4 – Software requirements specification (Lab 4), due 20.11.2026

Epic	Subtask	Assignee	Status
Write the SRS of the program	1. Choose a pattern for the SRS	Budkov O.	Open
	2. Write the purpose, definitions and system overview	Budkov O.	Open
	3. Write the overall description of the program	Budkov O.	Open
	4. Write the functional and non-functional requirements	Budkov O.	Open
	5. Update the repository and the issues on GitHub (Lab 4)	Budkov O.	Open
	6. Prepare the report of Lab 4	Budkov O.	Open

Table 3.5 – Iteration 5 – Testing of code (Lab 5), due 11.12.2026

Epic	Subtask	Assignee	Status
Refactor the program and cover it with unit tests	1. Move the calculation and the checks into separate functions	Budkov O.	Open
	2. Add exception handling for wrong input and files	Budkov O.	Open
	3. Write unit tests for the calculation of y and the input checks	Budkov O.	Open
	4. Update the repository and the issues on GitHub (Lab 5)	Budkov O.	Open
	5. Prepare the report of Lab 5	Budkov O.	Open

Task 3.5 Working with the Git Repository

The source code and the reports were added to the repository from the command line in the folder of the project: git init, git add ., git commit -m "Add code and reports of Labs 1-3", git branch -M main, git remote add origin https://github.com/oleksiibudkov/SE-Lab-Var3.git and git push -u origin main (Figures 3.11–3.12). After that, every change is made in small steps: the changed files are staged with git add, a commit with a short message is created with git commit and pushed to github.com with git push (Figures 3.13–3.14). A commit message can name the issue it finishes, for example “Add wireframe of the screens (closes #13)”, and GitHub closes this issue automatically.

For the work on a separate issue a new branch lab3-planning was created with git switch -c and pushed to github.com with git push -u origin lab3-planning (Figure 3.15). The changes are made in this branch and then merged into the main branch, so the main branch always has a working version of the program. The history of the commits is shown in Figure 3.16.

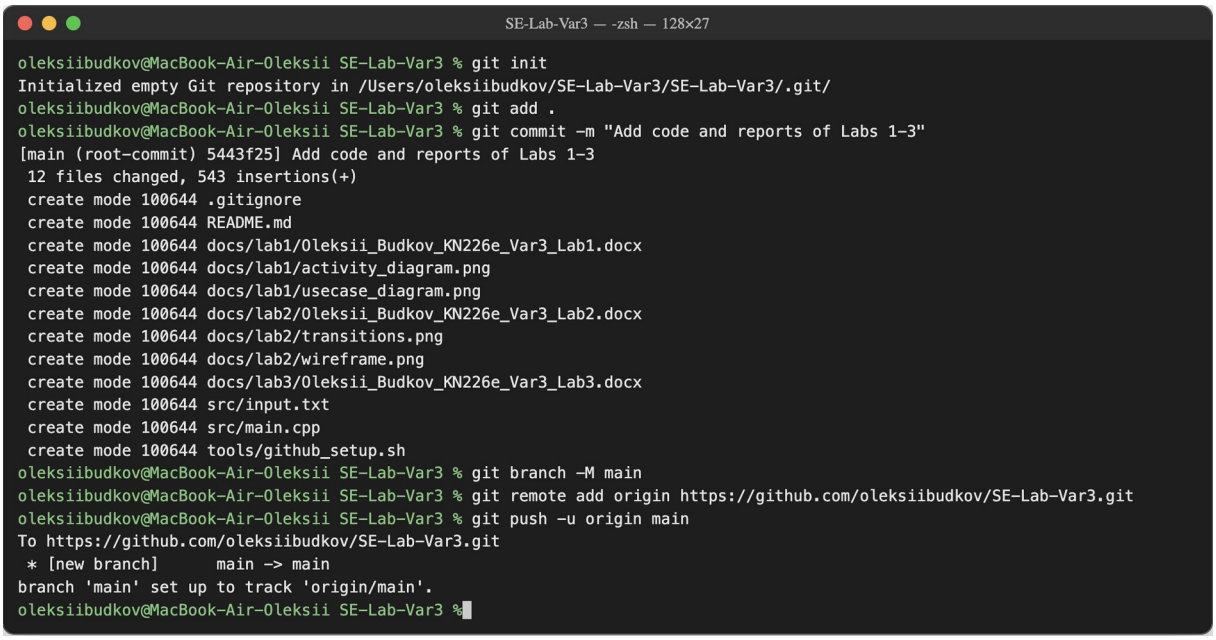


Figure 3.11 – Creating of a Git repository from the command line

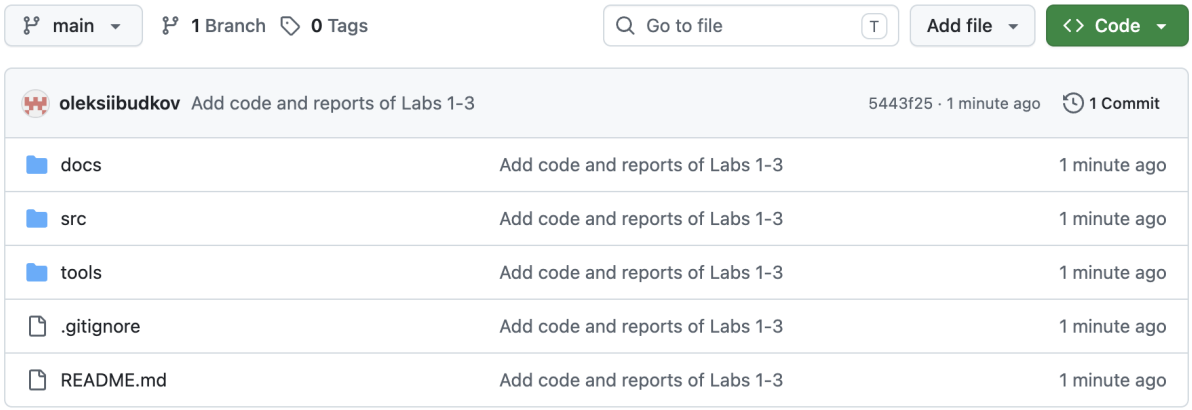


Figure 3.12 – Files of the project on github.com



Figure 3.13 – Committing of the changed file

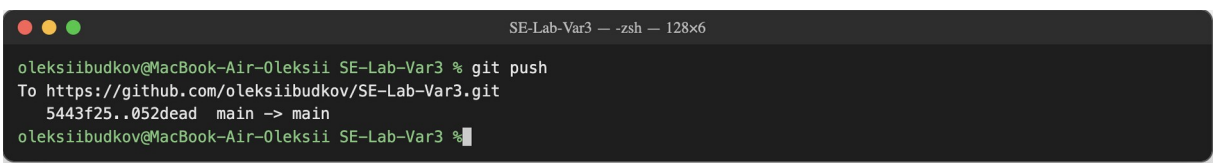


Figure 3.14 – Pushing of the commit

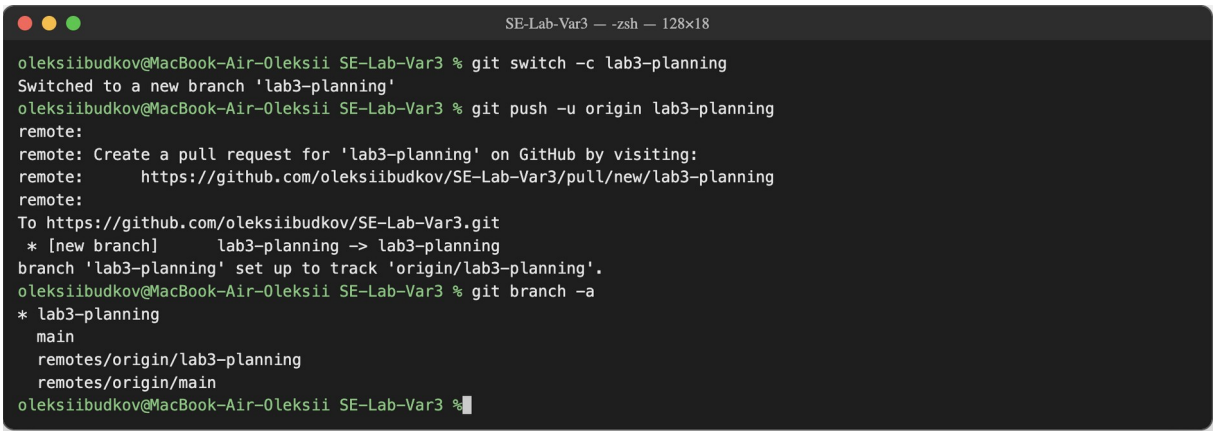


Figure 3.15 – Creating of a new branch

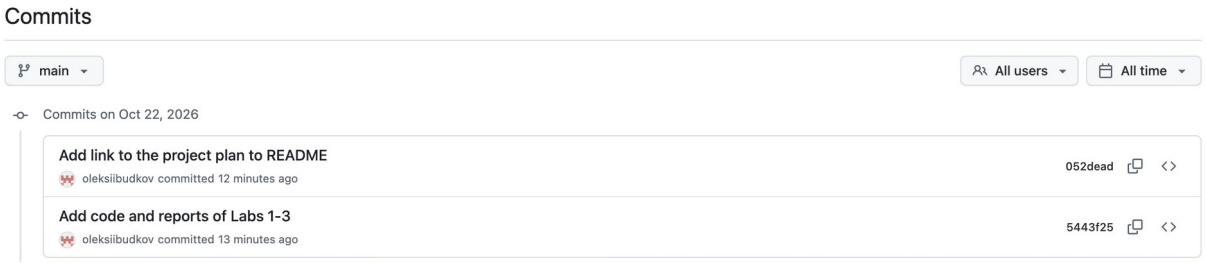


Figure 3.16 – History of commits

Conclusions

During this laboratory work we got skills and knowledge about working with the project tracking system GitHub. We registered an account, created a repository for the project and learned how to add collaborators to it. We chose an iterative life cycle model and planned the work of all laboratory works as five milestones with epic tasks and subtasks, marked them with labels and assigned them. We stored the code and the reports in the git repository from the command line, made commits, pushed them to github.com and used a separate branch for the work on an issue.